

# Lezione 6: Esercitazione Pratica

Collezioni, Interfacce Funzionali, Generics e Stream Avanzate

Michele Gentile

21 Maggio 2026

---

## 1 Motore di Validazione

1. Si definisca un'interfaccia funzionale generica personalizzata denominata **Valutatore** provvista di un solo metodo astratto: `boolean esamina(T target)`.
2. Si implementi la classe **FiltroDati** dotata di un metodo statico `applicaCriterio(List elementi, Valutatore criterio)` che restituisce una lista filtrata contenente solo gli oggetti che superano il test logico.
3. Si definisca la classe **Prodotto** con le proprietà **nome** (String), **prezzo** (double) e **disponibile** (boolean).
4. All'interno di una classe di test, si dimostri come invocare il metodo `applicaCriterio` su una lista di prodotti passando comportamenti dinamici espressi sia tramite **espressioni lambda** che tramite **method references** (es. filtrando i prodotti disponibili sotto una certa soglia di prezzo).

### 🔗 Obiettivo: Custom Functional Interfaces, Lambda e Method References

Comprendere l'architettura del software disaccoppiata basata sul polimorfismo parametrico e sulla programmazione funzionale, definendo tipi di target personalizzati.

### ⚠️ Attenzione: Uso dell'annotazione `@FunctionalInterface`

Sebbene non sia obbligatoria ai fini della compilazione del codice, ricordatevi di apporre sempre l'annotazione `@FunctionalInterface` sopra la dichiarazione delle vostre interfacce personalizzate. Questo costringerà il compilatore a segnalare un errore nel caso in cui venissero inseriti inavvertitamente altri metodi astratti.

## 2 Analizzatore di Testo e Frequenza Parole

1. Si crei una classe **AnalizzatoreTesto** che memorizza una lista di stringhe rappresentanti le righe di un testo letterario.

2. Sfruttando la versatilità delle pipeline ed i metodi avanzati della classe `Collectors`, si realizzi un metodo `mappaFrequenzaTermini()` volto a compiere le seguenti elaborazioni sequenziali:

- Suddividere ogni riga in singole parole (utilizzando espressioni regolari per isolare spazi e punteggiatura).
- Convertire tutti i termini estratti in caratteri minuscoli per evitare differenze di case-sensitivity.
- Scartare le stringhe vuote o i termini con lunghezza inferiore a 3 caratteri.
- Raggruppare i termini calcolandone la frequenza complessiva di occorrenza, restituendo una `Map<String, Long>`.

#### 🔗 Obiettivo: Tokenizzazione, FlatMap e Collectors.groupingBy

L'obiettivo risiede nell'integrare i processi di scomposizione delle stringhe in array con i meccanismi di riduzione avanzata e raggruppamento dei flussi di dati in mappe.

#### 💡 Tip: Raggruppamenti e Conteggi Concorrenti

Per contare con efficacia le occorrenze degli elementi all'interno di uno stream, combinate il metodo `Collectors.groupingBy` fornendo come secondo parametro (denominato downstream collector) il collettore speciale predefinito `Collectors.counting()`.

## 3 Monitoraggio della Rete di Sensori Territoriali

1. Si modelli la classe **Misurazione** caratterizzata dal valore della **temperatura** (double) e dal **timestamp** della lettura (String).
2. Si progetti la classe **SensoreMeteo** dotata di un codice identificativo univoco e di uno storico delle letture memorizzato in una `List`.
3. Si realizzi la classe **AreaGeografica** che aggrega al suo interno una collezione di oggetti di tipo *SensoreMeteo*.
4. Si scriva il metodo `calcolaMediaAlteTemperature(double soglia)` nella classe *AreaGeografica* che, estraendo tutte le misurazioni di tutti i sensori tramite **flatMap**, filtri i valori superiori alla *soglia* e ne calcoli la media aritmetica, gestendo l'eventuale assenza di dati tramite il tipo di ritorno `OptionalDouble`.

#### 🔗 Obiettivo: Pipeline Complesse, Aggregazione e Gestione dei Contenitori

Uso coordinato di operazioni strutturali annidate, manipolazioni condizionali su flussi di dati e gestione dei risultati potenzialmente vuoti mediante i tipi container.

#### ⚠️ Attenzione: Gestione Sicura di OptionalDouble

Il metodo `average()` invocato su un `DoubleStream` restituisce un'istanza di `OptionalDouble`. Non invocate mai direttamente il metodo `getAsDouble()` senza aver preventivamente verificato la presenza del dato tramite `isPresent()`, oppure prediligete l'approccio difensivo configurando un valore di fallback tramite `orElse()`.